

IST 615 – CLOUD MANAGEMENT

CONTAINERS & KUBERNETES

Jim Garrisi

Adjunct Professor

School of Information Studies

Syracuse University

Outline

2

- Announcements
- Containers, Kubernetes and OpenShift
- Lab #3 description

Announcements

3

□ Preparations for Final Project

- Goal: Use cloud services (at least two) to complete a “specific” task

Examples:

- Build a data analysis pipeline to process dataset X
- Automatically trigger a data analysis pipeline when a file is placed in cloud storage
- Send an alert via email when a sensor’s reading exceeds value Y
- Deploy a database that automatically logs some variable/changing information

Final Project

4

- ▣ You can work in a group – maximum 3 students
- ▣ Have a “task” defined for your project by March 17 @ 5pm
 - Submit via Blackboard a 2 to 4 page proposal detailing team members, project goals and cloud services/tools to be used plus any additional resources needed.
 - The description of cloud services you will use, must focus on how they support your solution, not on the generic description of what the service does as stated by the web/marketing material of the cloud service platform.
 - A diagram that illustrates the architecture of your cloud solution and/or the interaction between services (inputs / outputs) must be included in your proposal.
 - Links / references / sources of information to be used
 - Instructor might suggest that some smaller teams work together
 - **NOTE: For projects using AWS services... use a PERSONAL account, not an AWS Academy account!**
- ▣ Final Project due day of Final Exam

5

Recap

TCO

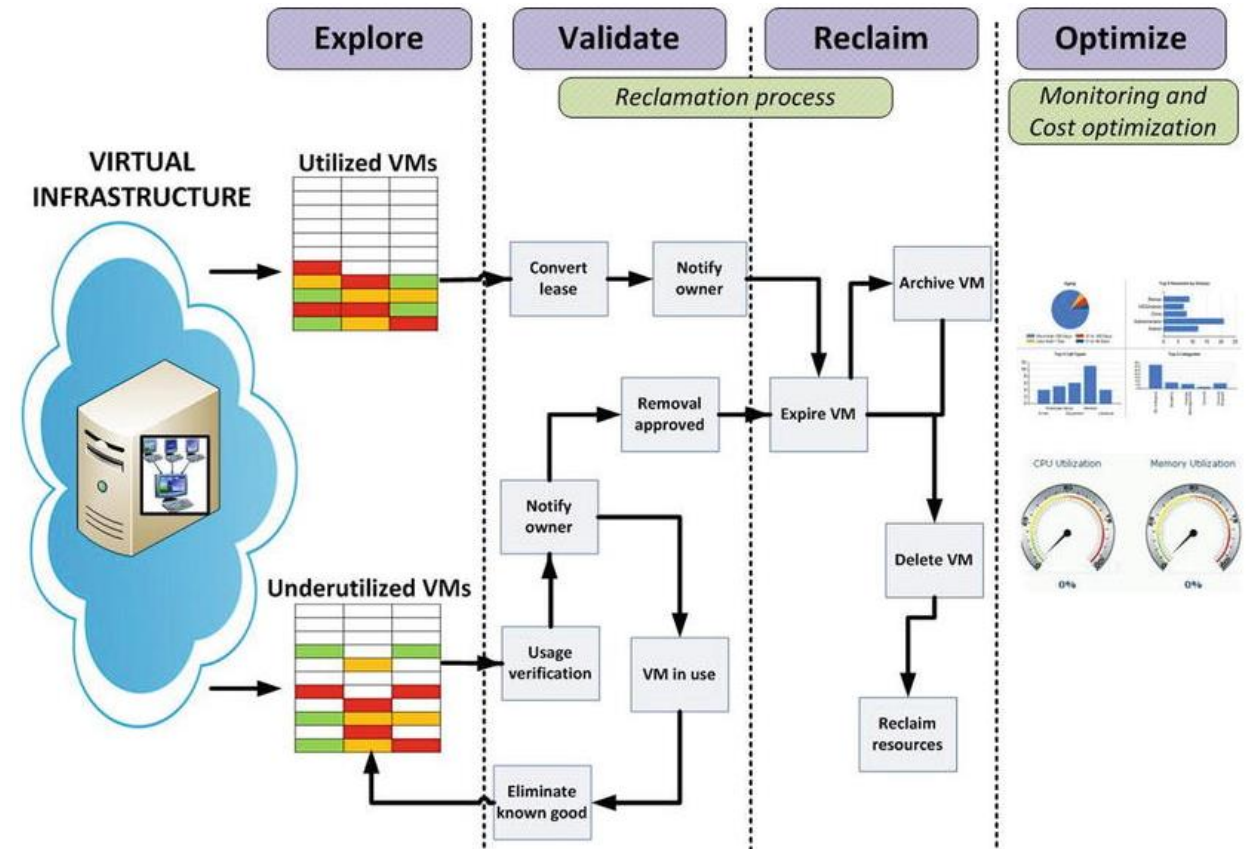
6

- ❑ Cloud's hidden costs
 - ▣ Service interruptions
 - ▣ Inappropriate service scaling
 - ▣ Denial of service attacks
 - ▣ Extra security
 - ▣ Disaster preparedness and recovery plan costs
 - ▣ Initial cost of cloud-readiness

Capacity management goals

7

- Efficient resource utilization
 - ▣ Manage workloads efficiently
- Reduce wasted resources
- Enable and support monitoring of service levels
- Controlling VM sprawl



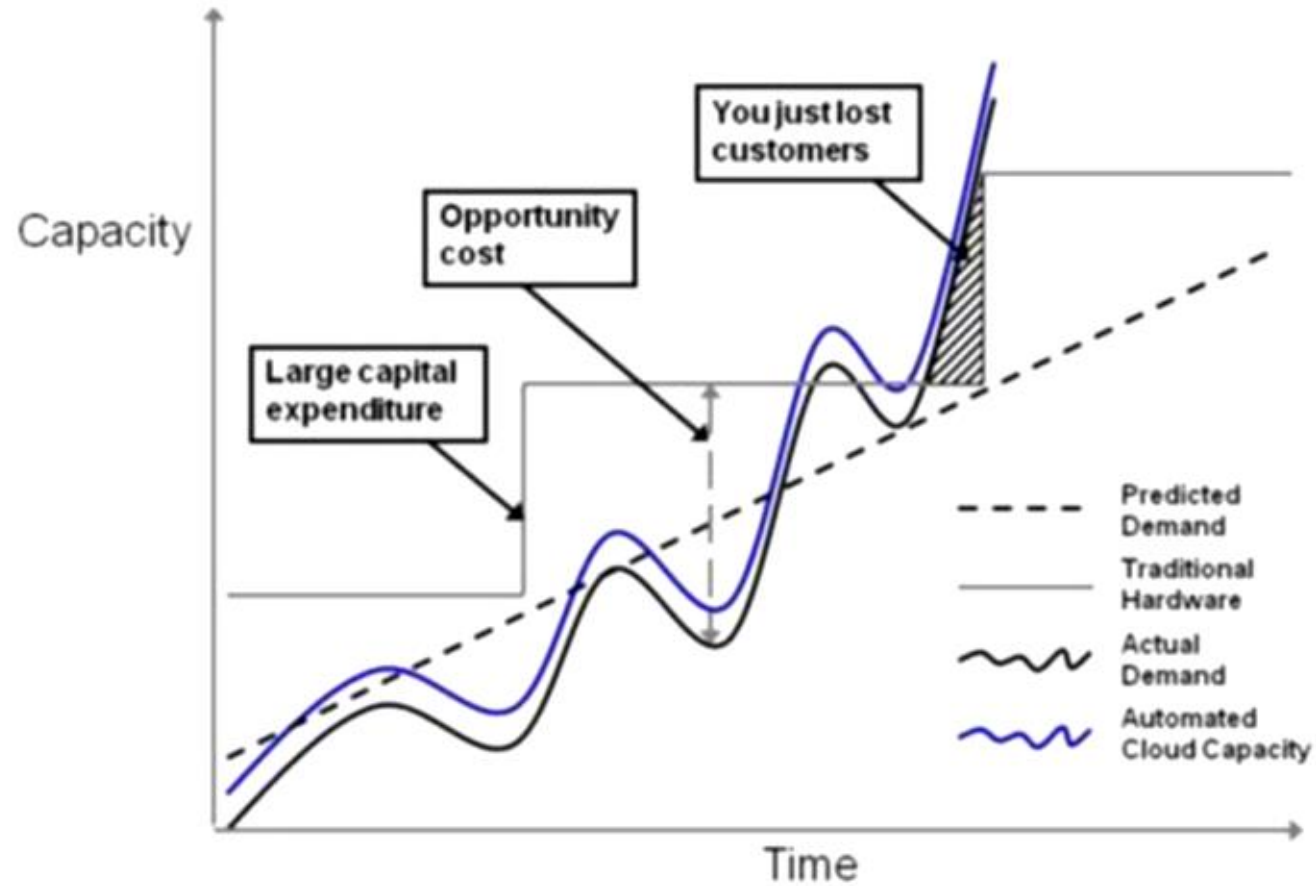
Capacity management goals (2)

8

- Define rules for handling resources that fail
- Define auto-scaling mechanisms to handle unforeseen demand and/or seasonal variations in demand
- Ensure the operation of cloud/multi-cloud resources under appropriate economic constraints

Capacity utilization

9



Based on illustrations by
Amazon Web Services

Auto-scaling

Design for Capacity

10

- ❑ Cloud solution architecture, performance targets and costs have to be analyzed until a viable design is determined
- ❑ The elasticity of the cloud moves capacity planning/design from a worst-case (peak) scenario perspective to an automatic scaling (of capacity) perspective
 - ▣ Use of a few large units of capacity (large VMs) vs. many small units of capacity (small VMs)

Types of pricing in the Cloud (AWS)

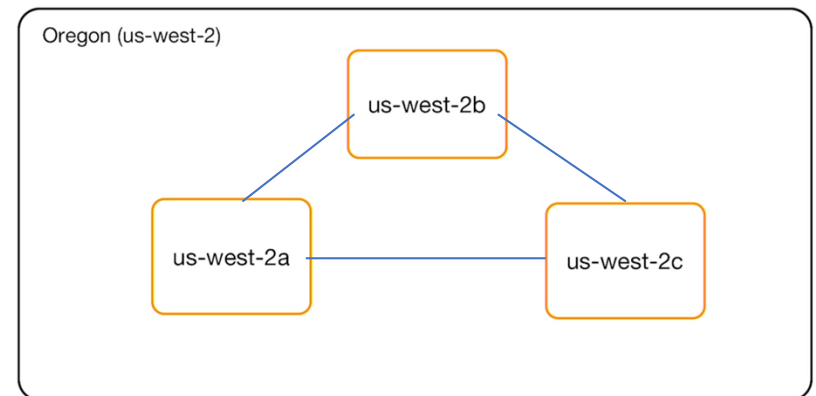
11

- On-Demand
- Reserved Instances
 - ▣ In AWS – Commitment levels: 1 year or 3 years
- Spot pricing
 - ▣ Bidding on spare capacity that the cloud provider has available
 - Can get up to 90% off the normal On-Demand price
 - ▣ Good for applications that have flexible start and end times
 - ▣ You don't pay more than your bid
 - You pay the market price until that price exceeds your bid
 - When the market price exceeds your bid, (in AWS), you have 2 minutes to shutdown your workload (or to migrate it).

AWS Regions

12

- https://aws.amazon.com/about-aws/global-infrastructure/regions_az/
- Each AWS Region consists of multiple, isolated, and physically separate Availability Zones within a geographic area.
 - ▣ Each AZ has independent power, cooling, and physical security and is connected via redundant, ultra-low-latency networks
 - ▣ Two to five AZs may be present in an AWS region
- Choosing between regions
 - ▣ Cost
 - ▣ Regulation (data sovereignty)
 - ▣ Latency



Azure -Regions

Azure offers more global regions than any other cloud provider with 60+ regions representing over 140 countries

- Regions are made up of one or more datacenters in close proximity.
- Provide flexibility and scale to reduce customer latency.
- Preserve data residency with a comprehensive compliance offering.



Understanding CPUs and vCPUs

14

- CPU cores vs threads

- ▣ <https://www.youtube.com/watch?v=hwTYDQ0zZOw>

- The mapping/definition of a vCPU is dependent on the cloud provider and the virtual machine instance type

- ▣ For EC2 general purpose instances (Spring/2021):

- Each vCPU is a thread of either an Intel Xeon core or an AMD EPYC core, except for M6g instances, A1 instances, T2 instances, and m3.medium.
 - Each vCPU on T4g and M6g instances is a core of the AWS Graviton2 processor.
 - Each vCPU on A1 instances is a core of an AWS Graviton Processor.

Auto Scaling

15

The autoscaling process can:

- Replace instances that have (performance) issues
 - ▣ Marked by the load balancer or hypervisor
 - ▣ Helps in building a self-healing infrastructure
- Maintain cluster size
- Grow cluster to meet increase in demand and scale down when needed

16

Containers, Kubernetes and OpenShift 101

Containers, Kubernetes, and OpenShift 101

Trends on the IT field related to containers

- ✓ DevOps
- ✓ CI/CD
- ✓ Microservices
- ✓ AI/ML
- ✓ Cryptocurrency
- ✓ Cloud-native
- ✓ Multi-cloud
- ✓ Hybrid cloud
- ✓ 5G
- ✓ Edge computing

What is a Container?

Distributing vs Running containers

Distributing and Running Applications as Containers

➡ **Container images** package an application together with all its dependencies, such as:

- ✓ System libraries
- ✓ Programming language runtimes
- ✓ Programming language libraries
- ✓ Configuration settings
- ✓ Static data files

➡ Container images are a **universal packaging and distribution mechanism**, independent of operating systems and programming languages.

➡ Container images serve as **blueprints for creating containers**.

Distributing and Running Applications as Containers

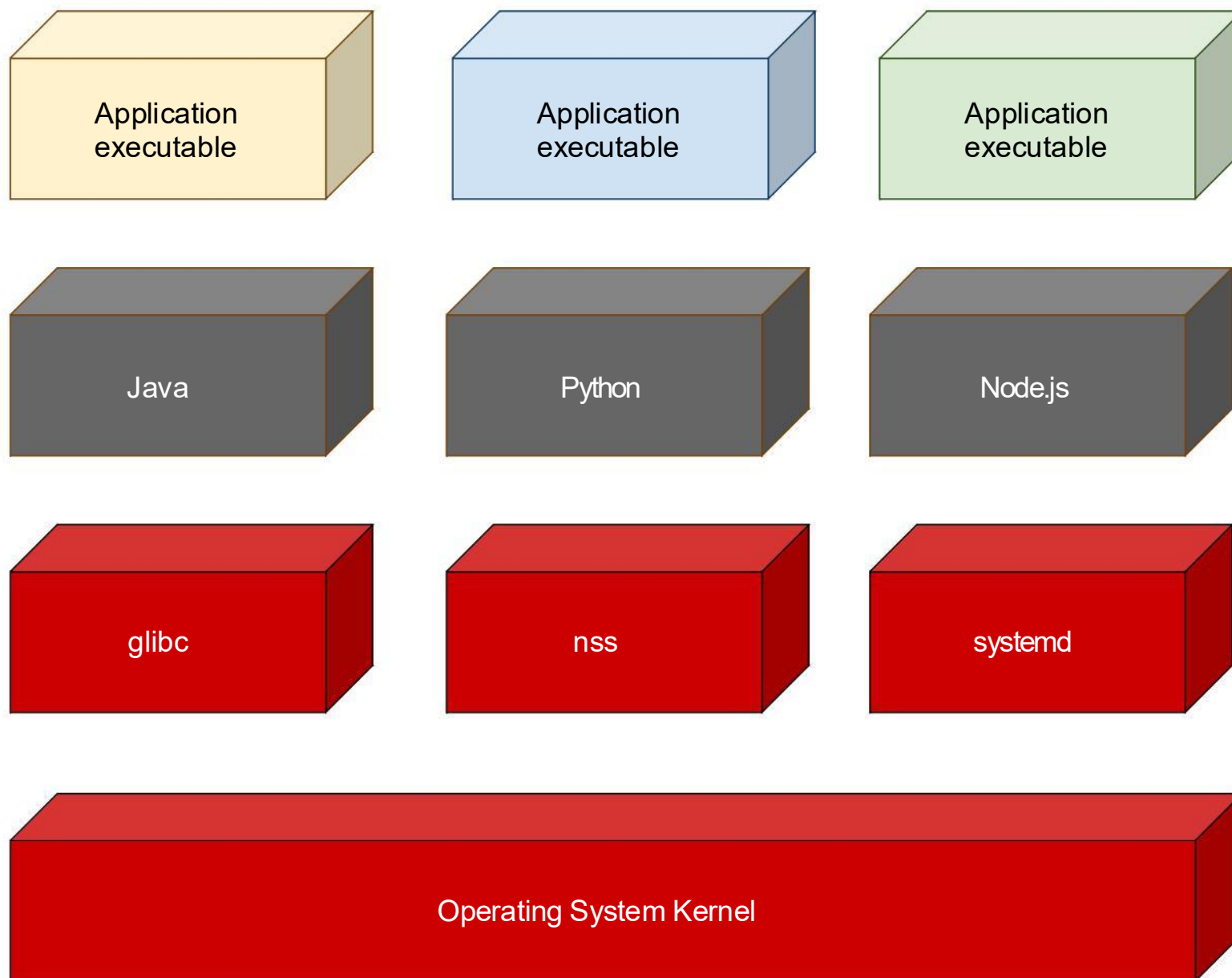
- Containers are operating system-level sandboxes that run applications isolated from each other and also from the core operating system services
- Containers create the illusion of an operating system dedicated to a single application and provide an exclusive set of operating system-managed resources, such as:
 - ✓ Processes
 - ✓ Filesystems (Files and folders)
 - ✓ Devices
 - ✓ Users and groups
 - ✓ Semaphores
 - ✓ Networks

Containers as a Packaging Mechanism



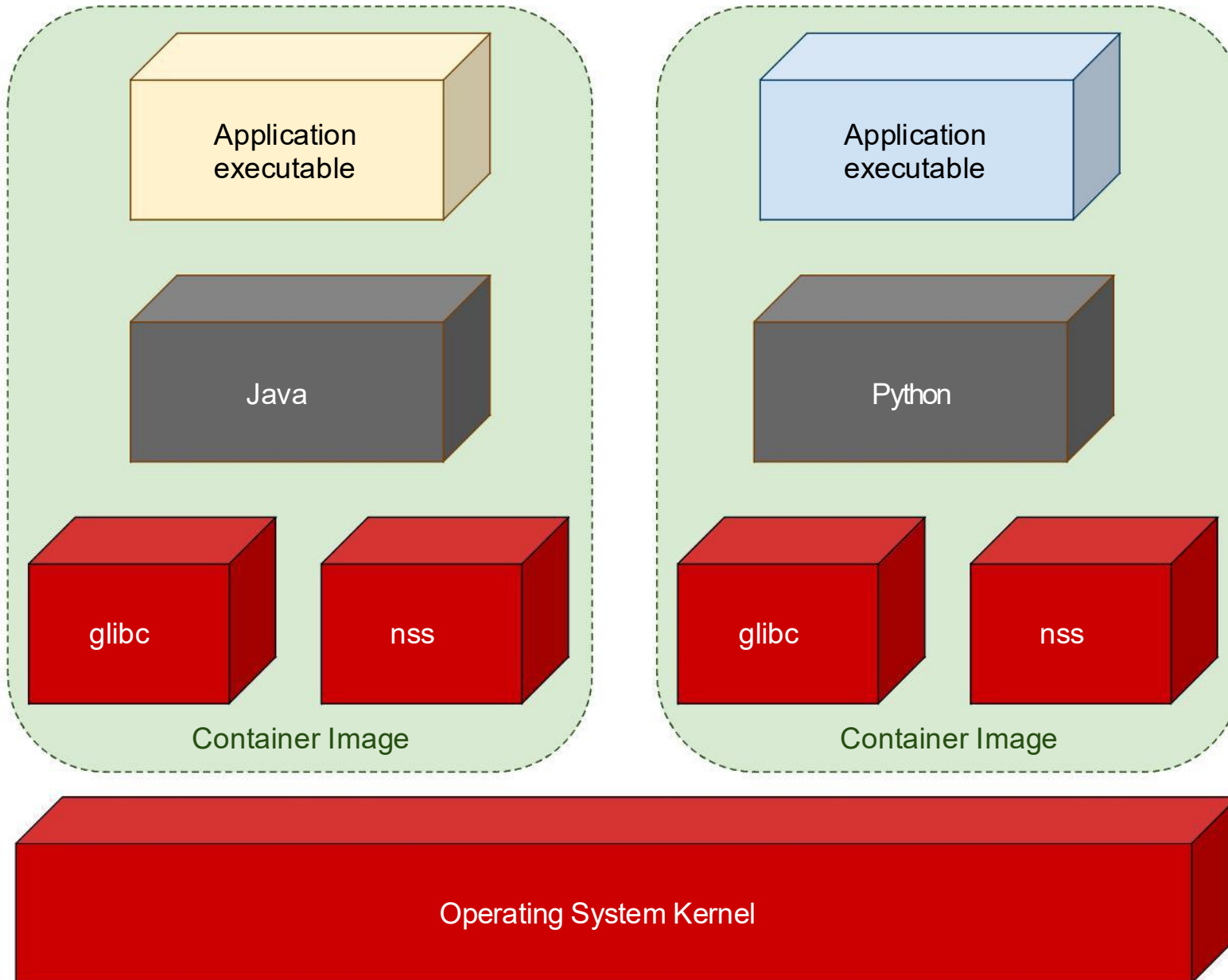
Containers are a standard way to **package** goods to ease **shipping** and delivery.

Software **containers** are a standard way to **package** applications to ease **deployment** and management.



Without containers, applications share a number of operating system-level libraries and services, and also a number of programming language-level runtimes and libraries.

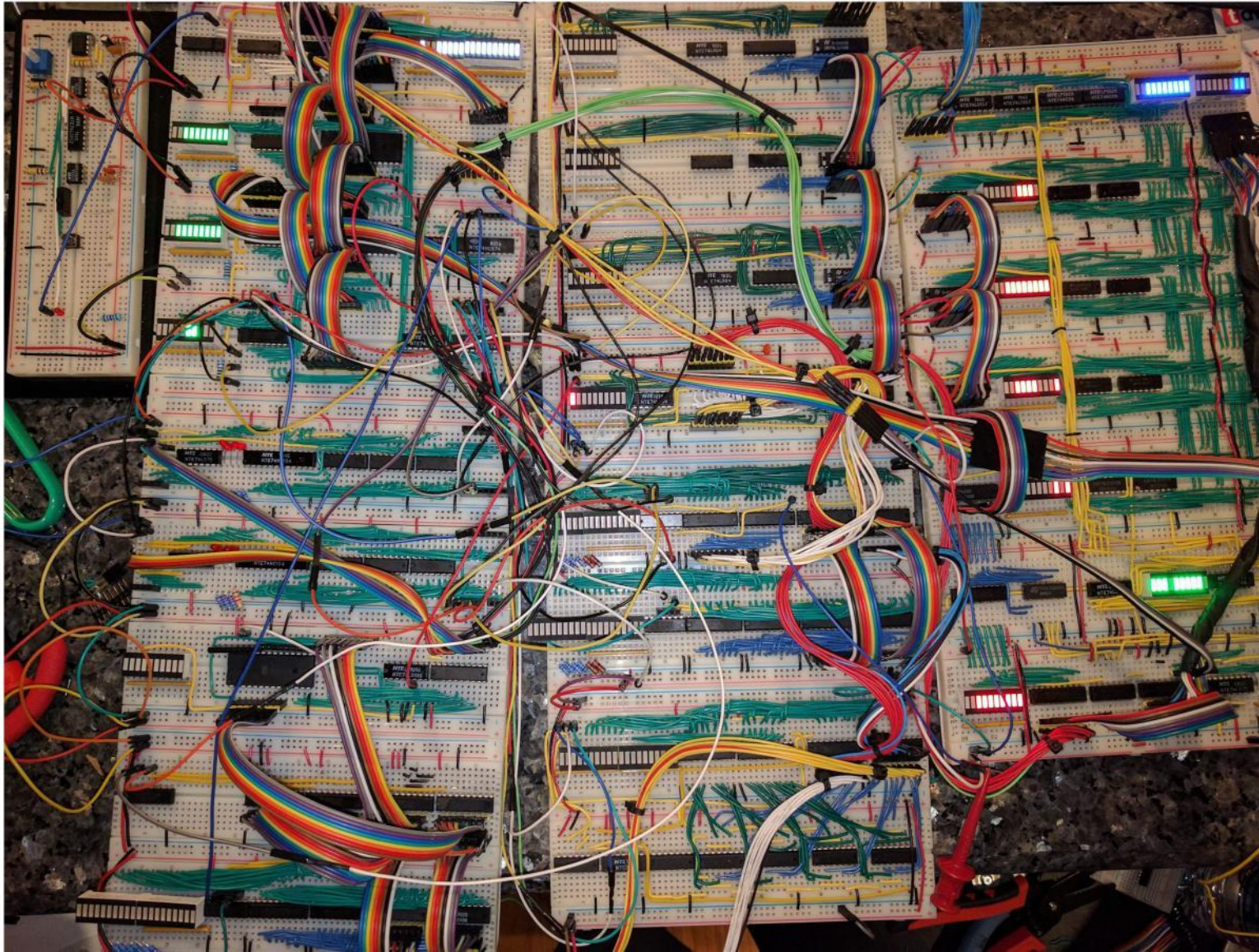
Making these shared libraries, services, and runtimes compatible with multiple applications on the same computer proved to be very hard.



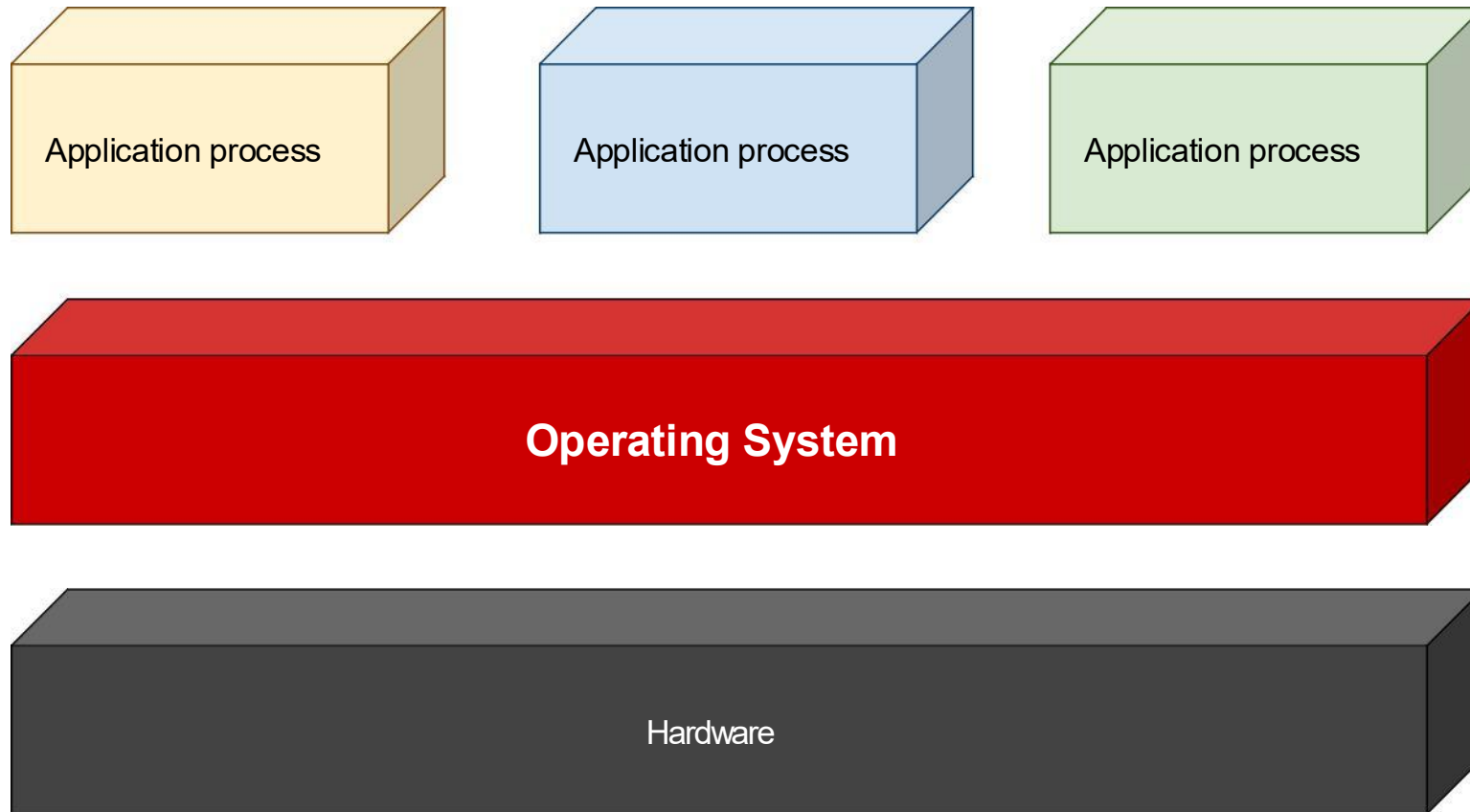
Containers images allow each application to bring its required dependencies without concern about conflicting dependencies from other applications.

A container image includes all programming language-level and also all user-mode, operating system-level dependencies of an application inside the same artifact that provides the application executable.

Containers as a Runtime Mechanism



In the beginning, applications had complete control over the hardware. There was no sandbox protecting an application.



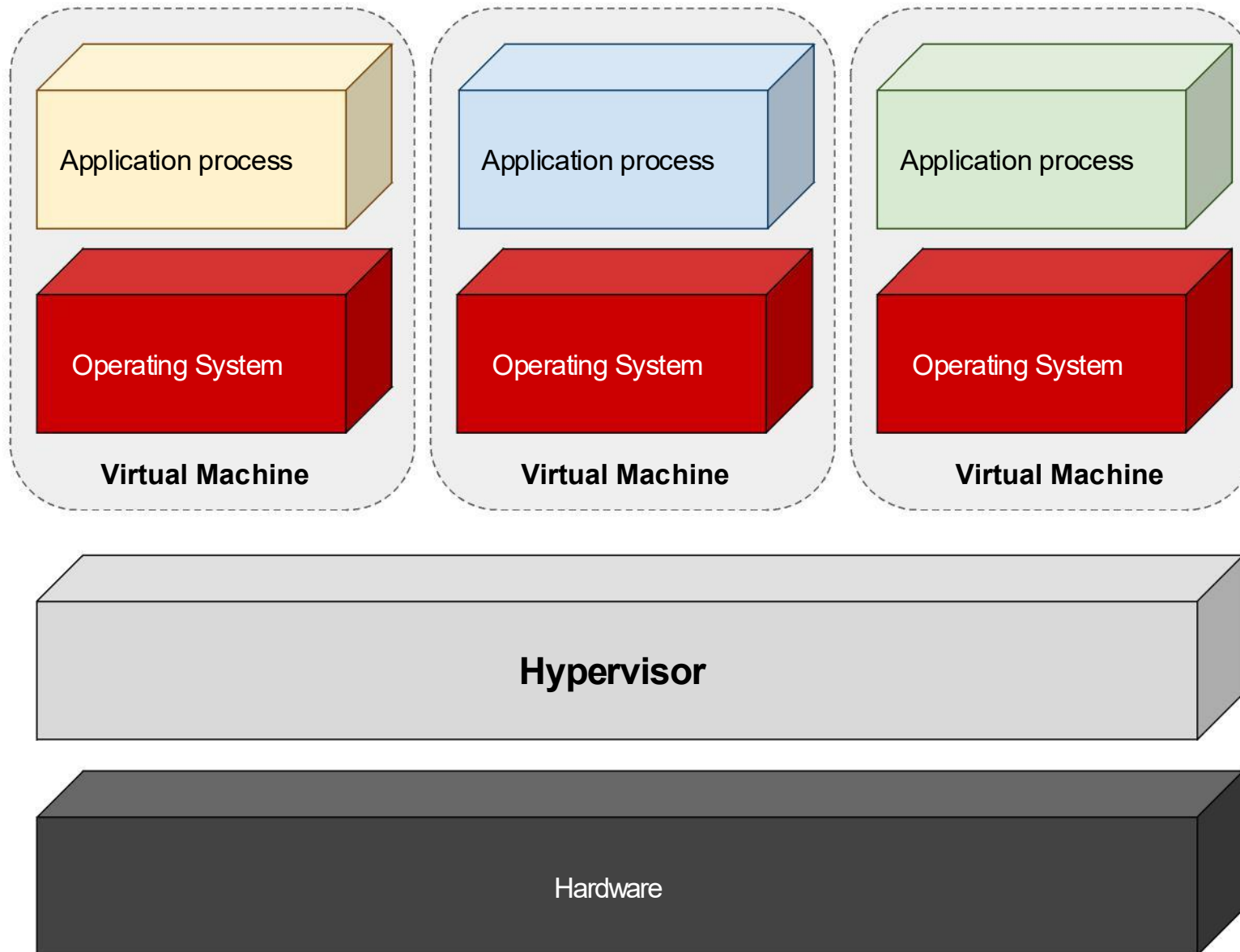
Then **operating systems** came and made applications run inside sandboxes called **processes** that prevented one application from accessing the memory of other applications.

Processes were also forced to release the CPU to other processes after a time slice.



The process sandbox was not strong enough. Applications could still interfere with each other and the operating system by indirect means.

The contact surface between applications was still too large.

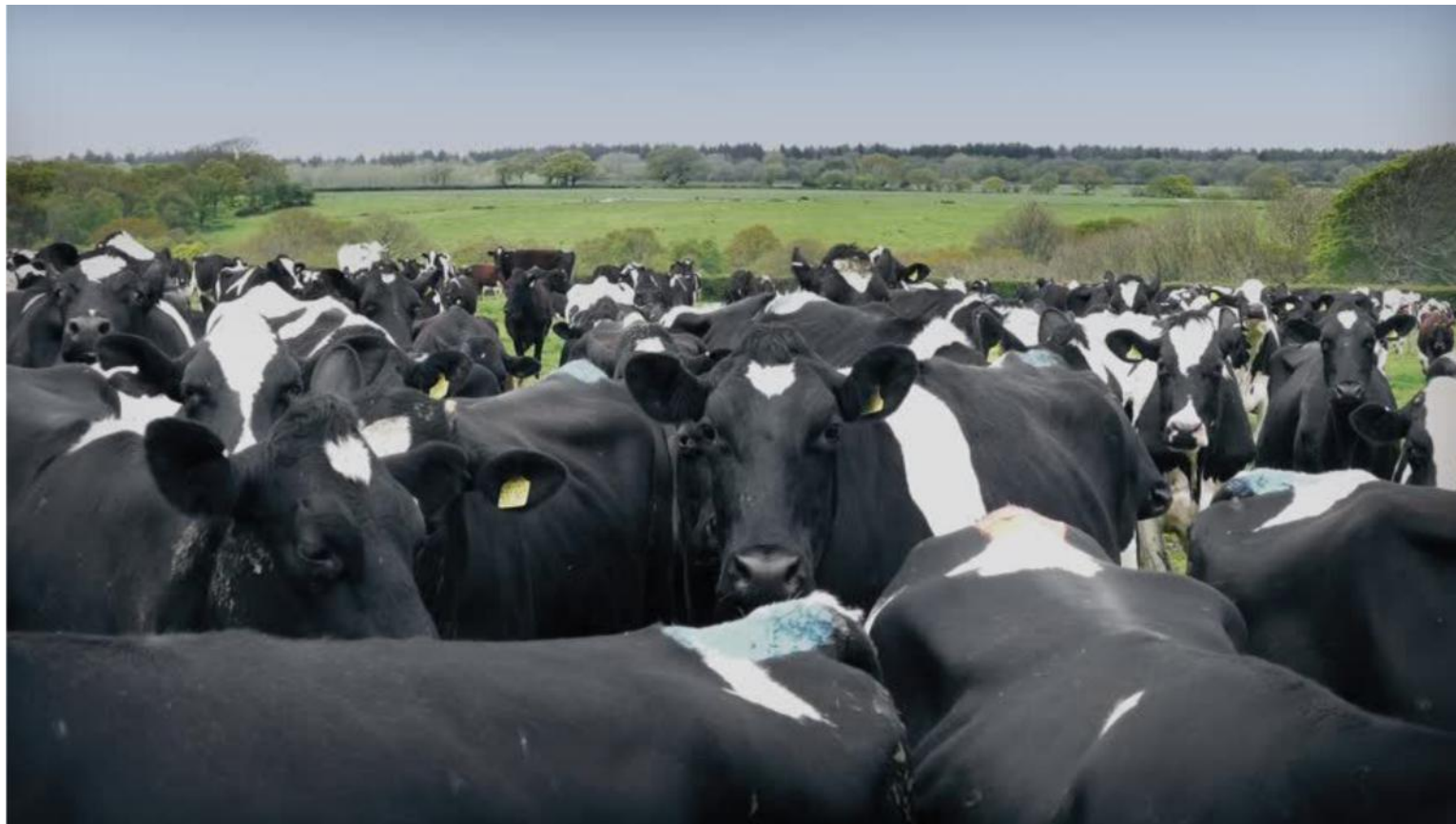


Then came the **hypervisor** and put both applications and their required operating systems inside sandboxes named **virtual machines**.



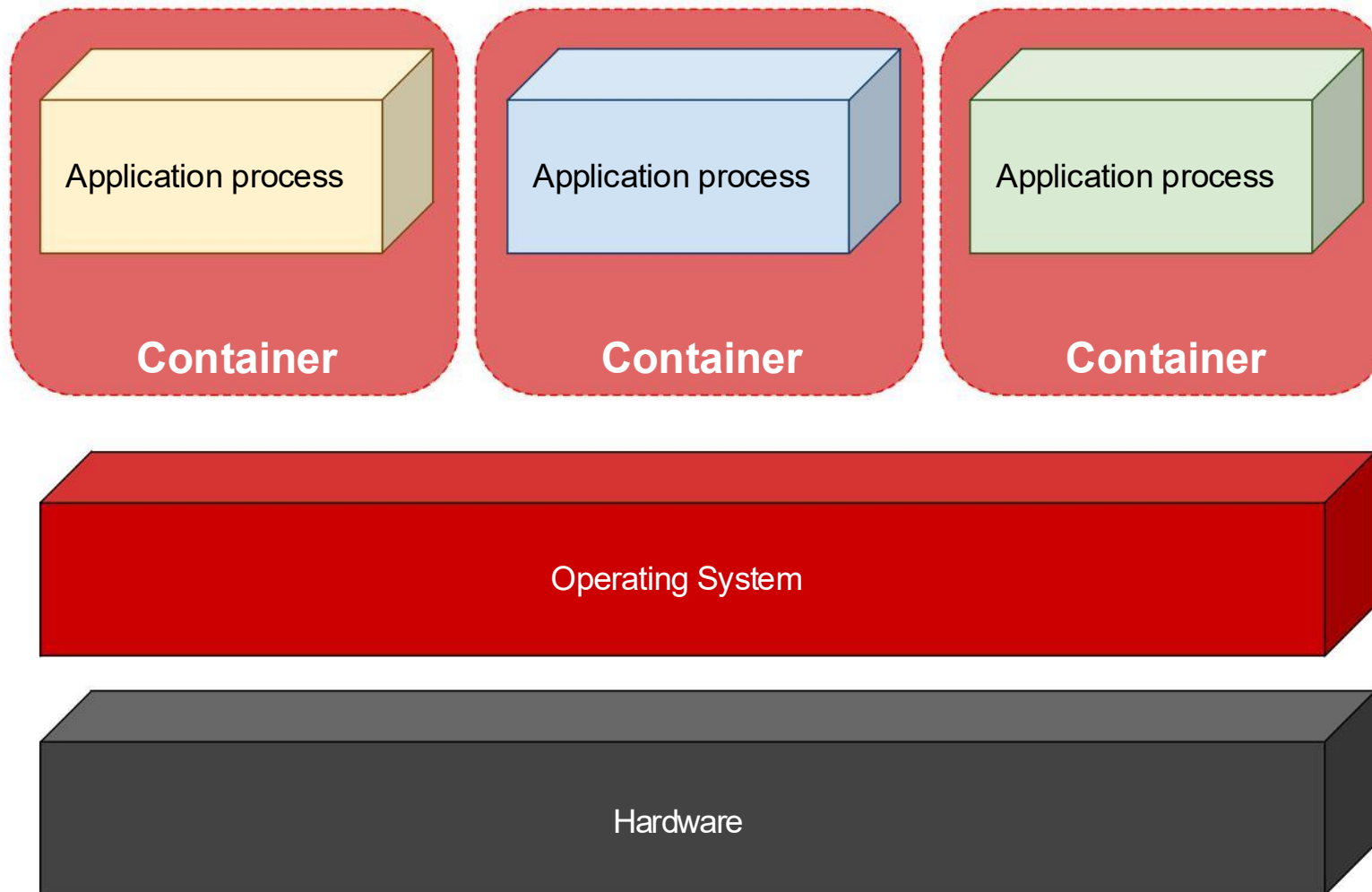
With more computers and more virtual machines, the larger cost factor changed from **computers** to **people**.

Also, the reliability of **people** instead of **software and hardware** become the largest risk factor.



The solution proposed by the cloud and agile practitioners is managing applications as **cattle** instead of not as pets.

Seeing all servers as indistinct from each other leads to easier automation and thus making people more scalable and reliable.



Containers provide stronger, faster, and lighter sandboxes for applications that you manage as cattle.

They also eliminate the need of managing an increasing number of machines.

Running Containers on My Computer

Container runtimes and **Container Engines** are just tools that ask the Linux kernel to start an application inside a container sandbox. There are many to choose from.

- Standards from the **Open Container Initiative** (<https://opencontainers.org>) allow interoperability between different tools that produce and consume the same containers and container images.
 - The Red Hat ecosystem of operating systems (Fedora, CentOS, and Red Hat Enterprise Linux) come with the *Linux Container Tools* app stream: **podman**, **skopeo**, and **buildah**.
- To use on your non-Linux work or personal machine the same tools you would use in a production RHEL system, either configure your own virtual machine or install **Code Ready Containers** (CRC) that does that for you.

Linux Container Tools

vs

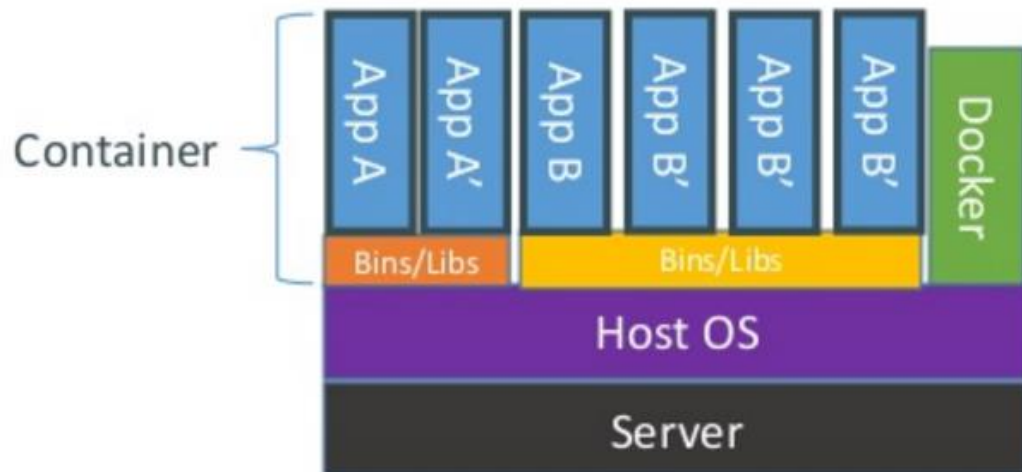
Docker

- ✗ Little known outside of the RHEL, CentOS, and Fedora ecosystems
- ✗ Still immature clients for Windows and Mac
- ✓ Set of single-purpose, standalone tools
- ✓ Easier to bug fix and secure, runs rootless
- ✓ Strong backwards-compatibility record and guarantees
- ✓ Strong community and sponsors

- ✓ Well-known among developers of all major platforms
- ✓ Good clients for Mac and Windows (daemon runs inside a Linux VM)
- ✗ Monolithic daemon that performs all tasks
- ✗ “Big ball of mud” issues to fix and secure, daemon has to run as root
- ✗ Incompatible changes pop-up by surprise (ask the Kubernetes developers)
- ✗ Sponsor is financially challenged and not very engaged with the community

Containers (1)

Containers are isolated,
but share OS and, where
appropriate, bins/libraries

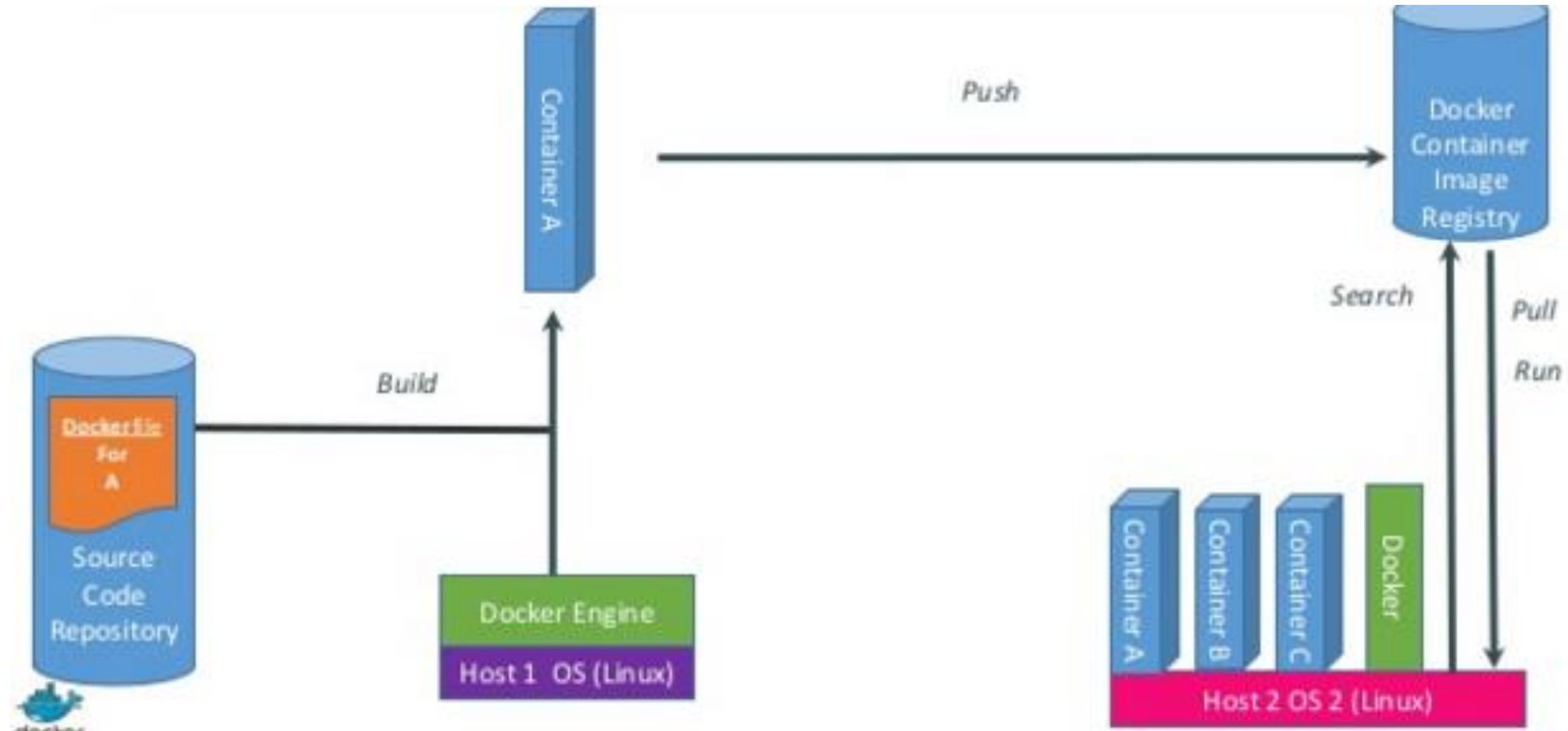


Containers allow users to
easily launch applications
on any virtual or physical
infrastructure,
**integrating all its
dependencies such as
code, runtime, system
tools and system libraries**

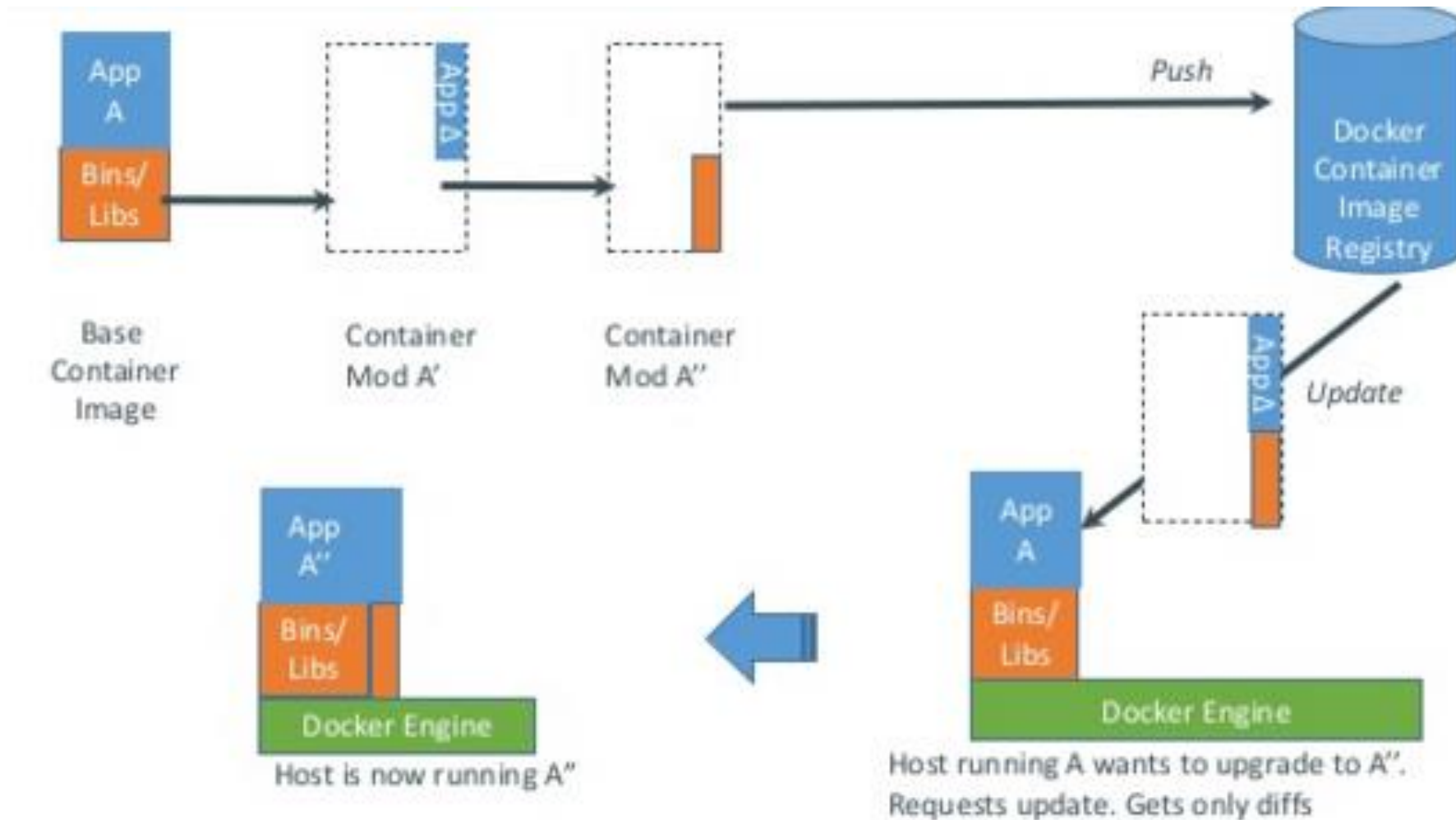
Video on containers

<https://www.youtube.com/watch?v=0qotVMX-J5s>

Containers (2)

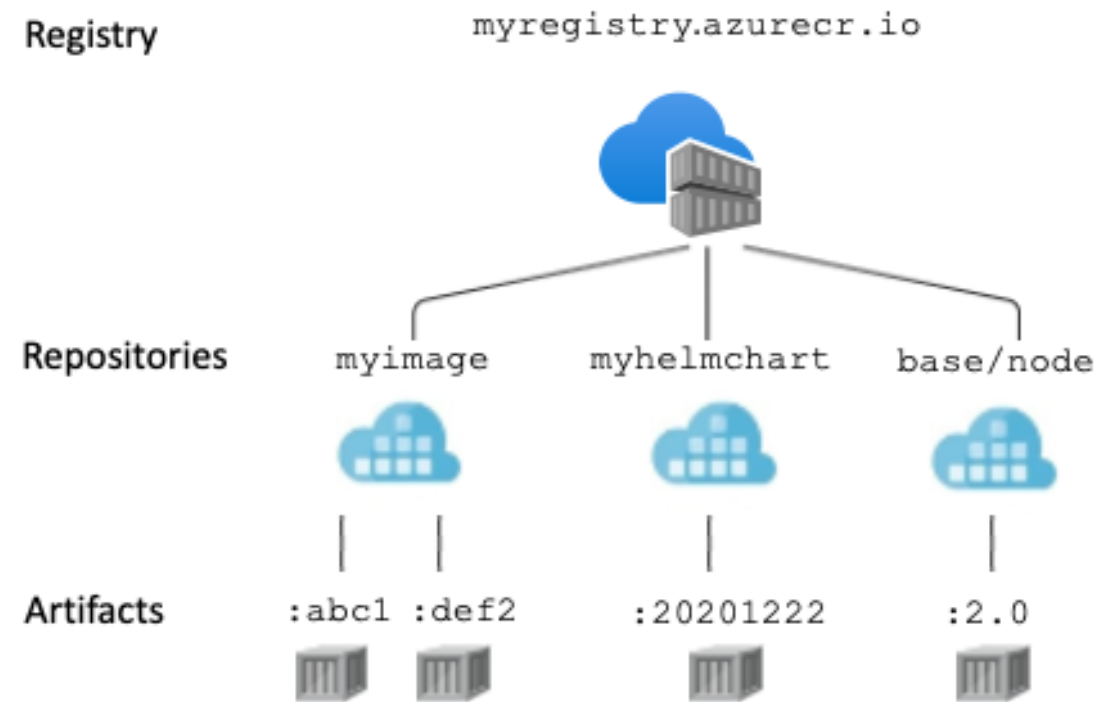


Container updates/changes



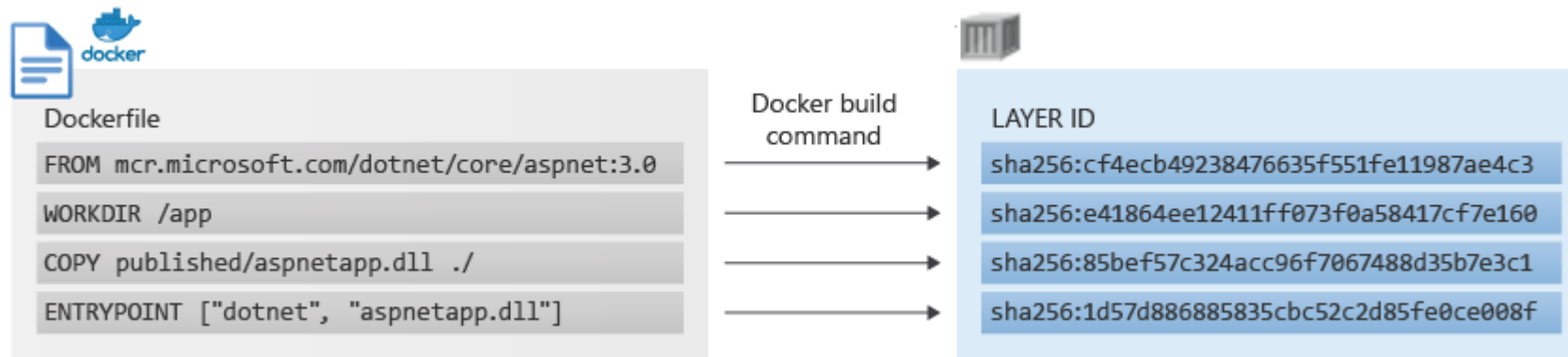
Managing containers – Introduction (1)

- You can push and pull container images to/from a container registry, which is something like a “GitHub” for container images.
- **Registry:** service that stores and distributes container images and related artifacts
 - E.g. Docker Hub, Quay.io, etc.
- **Repository:** Collection of container images or other artifacts (in a registry) that have the same name but different tags
- **Artifact:** container image, helm charts, manifests, etc.
- **Tag:** Alpha-numeric string that identifies an artifacts version



Managing containers – Introduction (2)

- Layer: Container images are made up of one or more layers. They define a stage in the container building process.



- Manifest: Information generated by the registry when the container is pushed to the registry. It uniquely identifies the artifacts and layers in the container.
- Manifest digest: Unique SHA-256 hash that identifies a manifests

Managing containers – Introduction (3)

Push by tag:

```
docker push myregistry.azurecr.io/samples/myimage:20210106  
docker push myregistry.azurecr.io/marketing/email-sender
```

Pull by tag:

```
docker pull myregistry.azurecr.io/marketing/campaign10-18/email-sender:v2
```

What is Kubernetes ?



Cloud and mobile applications are usually composed of multiple **microservices** that collaborate with each other.



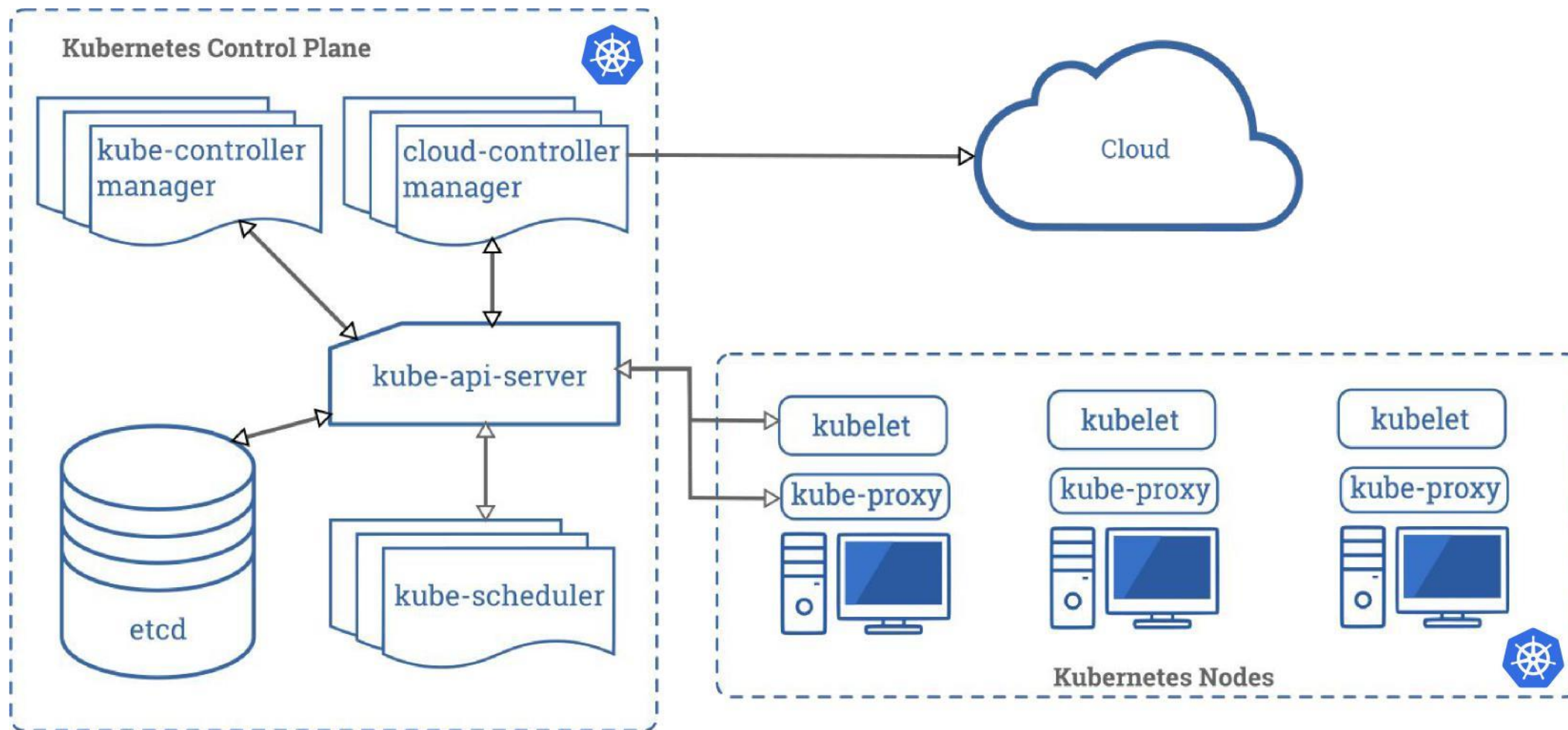
The more pieces you have,
the more things could go
wrong...



You need a good **helmsman** to take care of all containers on your ship.



You need a *good* **Kubernetes** to take care of all containers on your cluster of computers.



It looks harder than it actually is.

It is about *essential* complexity instead of *accidental* complexity

What Does Kubernetes Do for You?

- ✓ Find a computer in a cluster with sufficient available capacity to run a container for an application.
- ✓ Route network connections between applications to the right container no matter in which computer it is running.
- ✓ Run more or less containers of the same application according to demand (scaling).
- ✓ Load balance network connections among multiple containers of the same application.
- ✓ Start a new container to replace a failed container (or all containers from a failed computer).
- ✓ Re-attach applications to its data when their containers restart in different computers.

<https://www.youtube.com/watch?v=IMOZCDhH7do>

What is OpenShift ?

OpenShift is Red Hat's enterprise distribution of **Kubernetes**

Red Hat Advanced Cluster Management for
Kubernetes

Multicloud management

Observability : Discovery : Policy : Compliance : Configuration : Workloads

Red Hat Advanced Cluster
Security for Kubernetes

Advanced security

Declarative security : Vulnerability management : Network segmentation : Threat detection & response

OpenShift Container
Platform

Manage workloads

Build cloud-native apps

Data-driven insights

Developer productivity

Platform services

Service mesh : Serverless
Builds : CI/CD pipelines
Log management
Cost management

Application services

Languages & runtimes
API management
Integration
Messaging
Process Automation

Data services

Databases : Cache
Data ingest & prep
Data analytics : AI/ML
Data mgmt & resilience

Developer services

Developer CLI : IDE
Plugins & extensions
CodeReady Workspaces
CodeReady Containers

OpenShift
Kubernetes Engine

Kubernetes cluster services

Automated Ops : Over-the-air updates : Monitoring : Logging : Registry : Networking : Router : Virtualization : OLM : Helm

Kubernetes (orchestration)

Linux (container host OS)

Here is **Kubernetes**

Red Hat Advanced Cluster Management for
Kubernetes

Red Hat Advanced Cluster
Security for Kubernetes

OpenShift Container
Platform

OpenShift
Kubernetes Engine

Multicluster management

Observability : Discovery : Policy : Compliance : Configuration : Workloads

Advanced security

Declarative security : Vulnerability management : Network segmentation : Threat detection & response

Manage workloads

Build cloud-native apps

Data-driven insights

Developer productivity

Platform services

Service mesh : Serverless
Builds : CI/CD pipelines
Log management
Cost management

Application services

Languages & runtimes
API management
Integration
Messaging
Process Automation

Data services

Databases : Cache
Data ingest & prep
Data analytics : AI/ML
Data mgmt & resilience

Developer services

Developer CLI : IDE
Plugins & extensions
CodeReady Workspaces
CodeReady Containers

Kubernetes cluster services

Automated Ops : Over-the-air updates : Monitoring : Logging : Registry : Networking : Router : Virtualization : OLM : Helm

Kubernetes (orchestration)



Linux (container host OS)

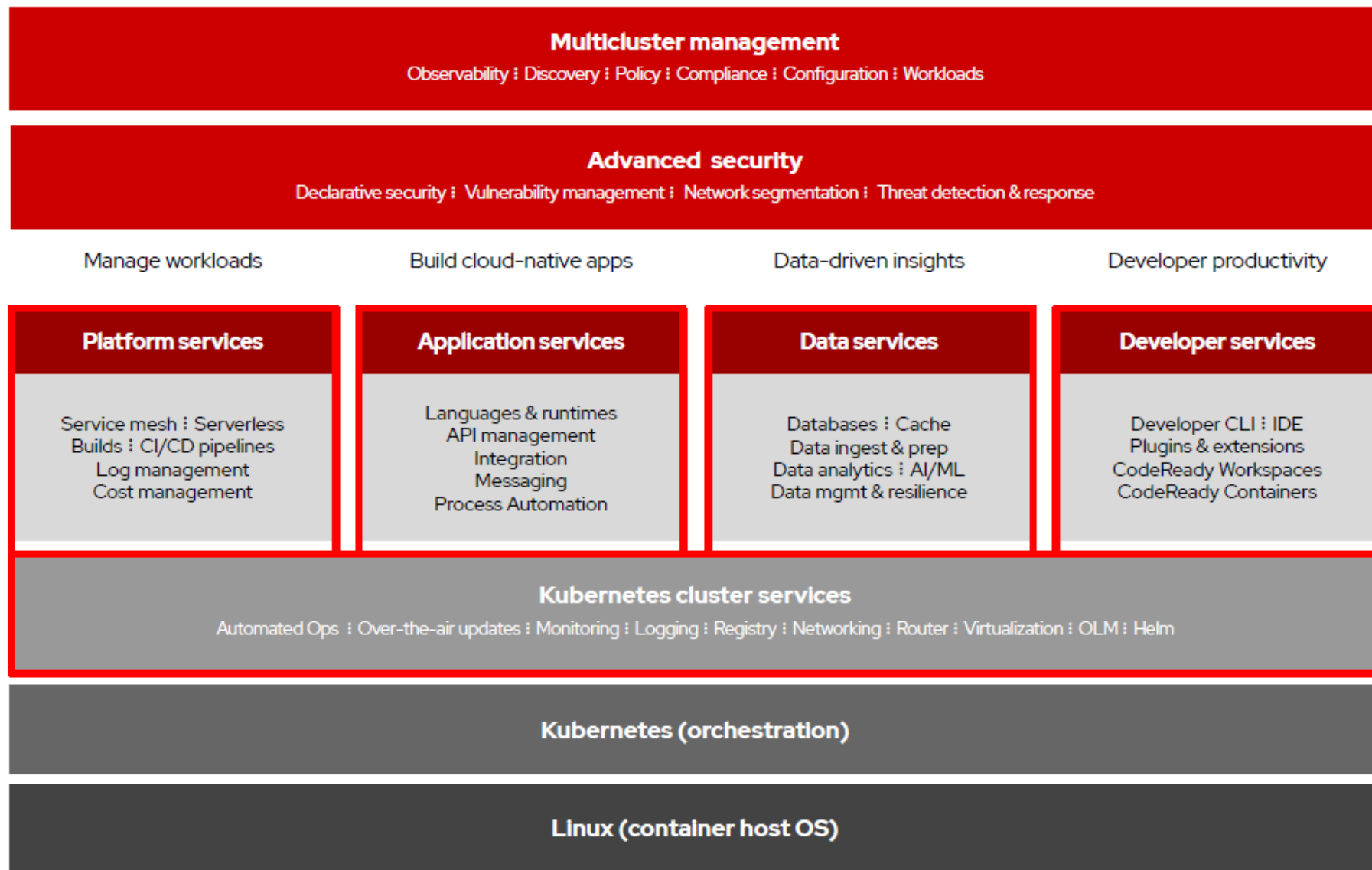
OpenShift adds all this

Red Hat Advanced Cluster Management for
Kubernetes

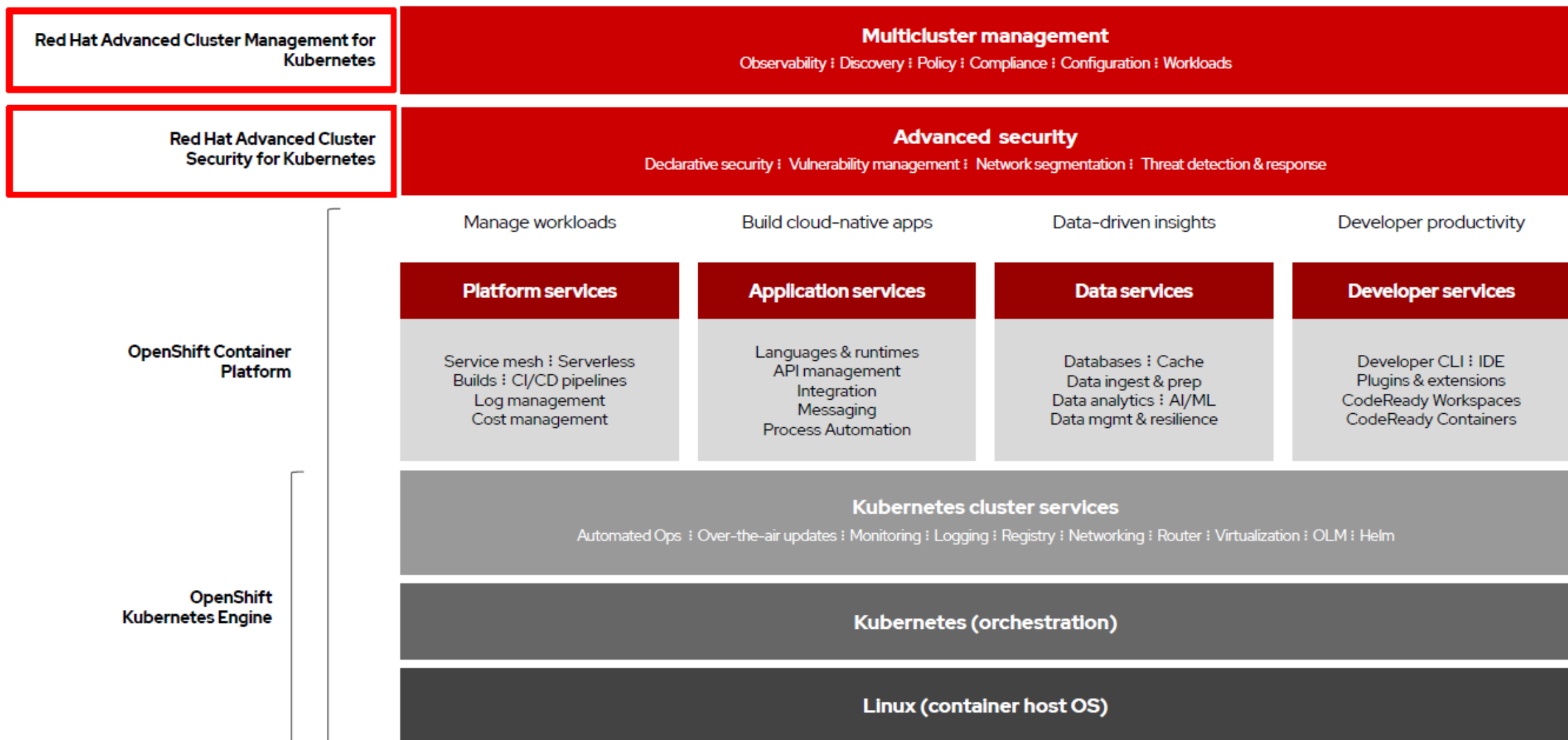
Red Hat Advanced Cluster
Security for Kubernetes

OpenShift Container
Platform

OpenShift
Kubernetes Engine



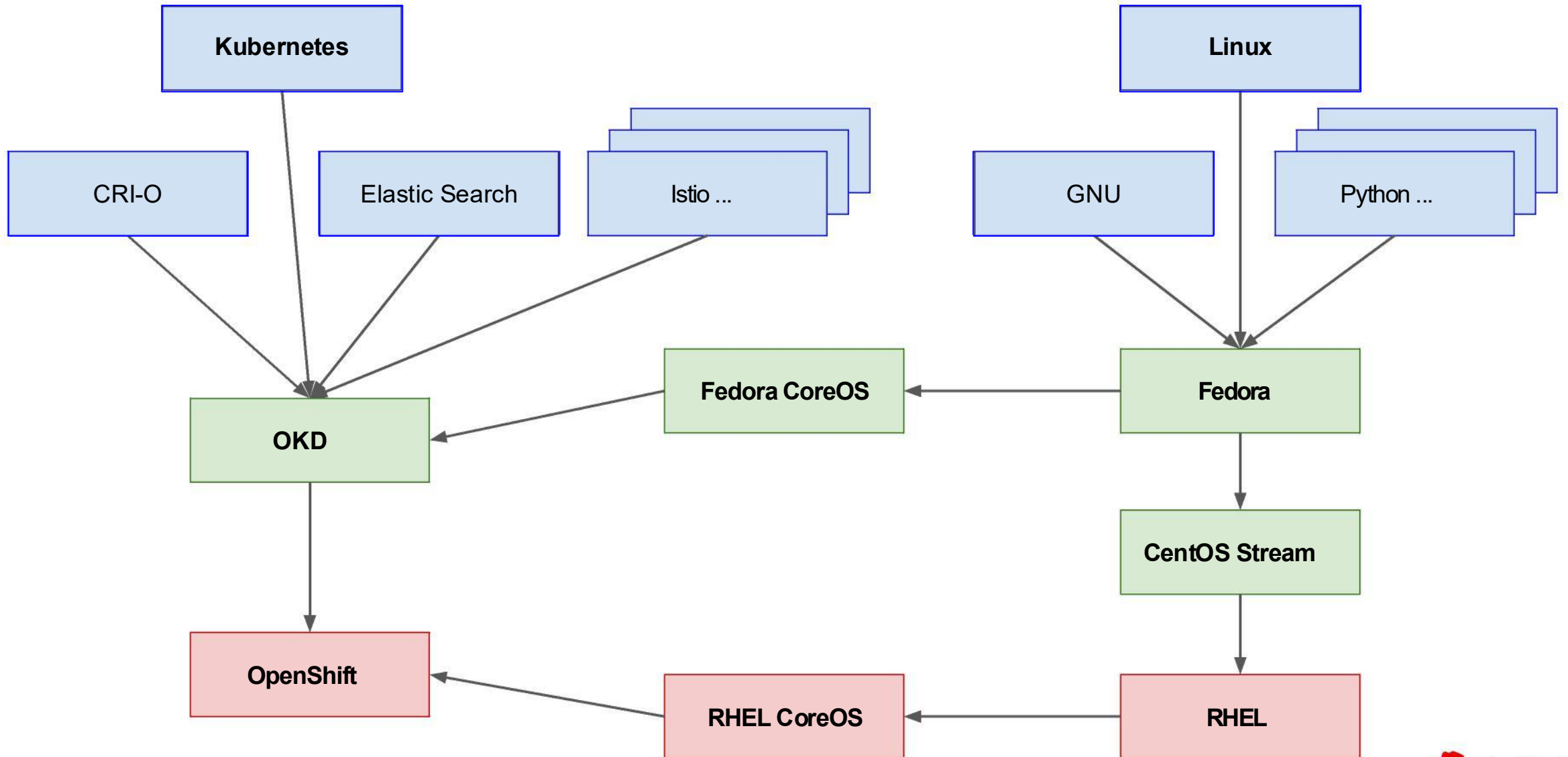
OpenShift is also a family of add-on products



What OpenShift Does for You Beyond Pure Kubernetes

- ✓ Provide a registry server to store and distribute container images.
- ✓ Build container images from source code and CI/CD pipelines.
- ✓ Add strong security controls based on roles and authentication providers.
- ✓ Add centralized logging, monitoring, alerts, and metrics.
- ✓ Manage installation and patching of the operating system on computers in the cluster.
- ✓ Route and load balance network connections from outside the cluster to the correct application container inside the cluster.
- ✓ Integrate with network and storage providers for popular virtualization, cloud, and hardware vendors.

From community to commercial



Lab #3 Description

56

- AWS S3 (Simple Storage Service)
 - ▣ https://www.youtube.com/watch?v=114_sXHO8U
- AWS IAM (Identity and Access Management)
 - ▣ <https://www.youtube.com/watch?v=UI6FW4UANGc>
- Lab 3 posted in Blackboard
 - ▣ Due March 17
 - ▣ Use of AWS S3 storage service
 - ▣ Integrating two AWS services to work together (EC2 & S3) via IAM
 - ▣ Additional details